

Laboratorio 2: Programación GPGPU con CUDA

Simulador gravitatorio N-cuerpos en 2D (C++/CUDA)

Departamento de Ingeniería Informática
Universidad de Santiago de Chile

6 de julio de 2026

1. Objetivos

Este laboratorio continúa el simulador N-cuerpos del Laboratorio 1. El modelo físico, las clases de dominio y el pipeline de CI ya están resueltos; aquí el foco es portar el núcleo computacional a GPU con CUDA, reforzar prácticas de ingeniería con Git y automatizar tareas repetitivas mediante agentes de IA. Al finalizar, ustedes serán capaces de:

- Implementar kernels CUDA para un problema de simulación con costo $\mathcal{O}(N^2)$
- Gestionar memoria host/device, transferencias y layout de datos (SoA) orientado a coalescing
- Usar `__shared__`, `__syncthreads()`, reducción paralela y operaciones atómicas en GPU
- Validar resultados GPU frente a una referencia CPU con tolerancia en coma flotante justificada
- Analizar el impacto de transferencias host-device en speedup y ley de Amdahl
- Aplicar un flujo Git profesional: ramas protegidas, releases, changelog e issues vinculados a MR/PR
- Configurar agentes de IA para documentación, revisión de bugs y revisión de merge requests
- Discutir cuándo un cambio es mecánico (aptitud para agente) y cuándo requiere intervención humana

2. Relación con el Laboratorio 1

Se asume que el grupo trabaja sobre el **mismo repositorio** del Laboratorio 1 (continuidad del historial Git). Solo se acepta fork si queda documentado en el README con URL, motivo y cómo se mantiene trazabilidad con el Lab 1. **No deben reimplementar** la física, el integrador Euler ni la estructura general de clases desde cero; refactor mínimo solo si queda justificado en un issue y en el reporte.

Lo que **sí** deben agregar o adaptar:

- Kernels CUDA para `computeAccelerations` (variante básica y variante con memoria compartida)
- Capa de buffers y transferencias host/device
- Tests de equivalencia CPU (referencia serial del Lab 1) vs. GPU

- Flujo Git riguroso, `CHANGELOG.md` y agentes de IA en CI

La versión CPU serial del Lab 1 debe conservarse como **baseline de corrección** y para benchmarks comparativos.

3. Organización del equipo (cinco integrantes)

Cada grupo tiene cinco integrantes. Cada persona asume un rol principal. En el `README.md` y en el reporte incluyan una tabla con nombre, rol y responsabilidades concretas. En el test de salida del curso se les podrá preguntar por el trabajo asociado a su rol, incluyendo issues y MR/PR que hayan abierto o revisado.

Rol	Responsabilidades típicas
1. Kernels CUDA	Kernels <code>computeAccelerationsKernel</code> (un hilo por cuerpo i , bucle interno sobre j) y <code>computeAccelerationsKernelShared</code> ; lanzadores <code>host</code> ; <code>CUDA.CHECK</code> ; convención de índices y protección de bordes.
2. Host/device y memoria	<code>CudaBuffer</code> o equivalente RAII; layout SoA en device (<code>d_mass</code> , <code>d_x</code> , <code>d_y</code> , ...); <code>cudaMalloc/cudaMemcpy/cudaFree</code> ; minimizar copias por paso temporal; sincronización con <code>cudaDeviceSynchronize</code> donde corresponda.
3. Integración y validación	Integración Euler en host tras sincronizar el device (kick/drift sobre buffers copiados o ya en host); tests CPU vs. GPU con tolerancia documentada en README; métricas K y U en GPU con reducción en <code>__shared__</code> y una variante con <code>atomicAdd</code> .
4. Git, releases y agentes	Protección de rama <code>main</code> ; ramas <code>feature/*</code> y <code>fix/*</code> ; <code>CHANGELOG.md</code> ; issues en GitHub/GitLab creados y etiquetados por el equipo; configuración y documentación de los tres agentes de IA (ver sección 7).
5. Calidad, CI y visualización	Extender CI del Lab 1 (compilación + <code>make test</code> en cada MR); revisión humana de issues y MR/PR; Docker con imagen CUDA; gráficos de speedup, estudio <code>blockDim.x</code> y trayectorias a partir de datos del clúster.

4. Descripción del problema

El problema físico es el mismo que en el Laboratorio 1: simulador gravitatorio de N cuerpos en 2D, ley newtoniana todo-pares con suavizado Plummer $\varepsilon > 0$, integración Euler explícita, costo $\mathcal{O}(N^2)$ por paso. No se solicitan Barnes–Hut ni PM/PP.

La aceleración del cuerpo i es

$$\vec{a}_i = G \sum_{j \neq i} m_j \frac{\vec{r}_j - \vec{r}_i}{(\|\vec{r}_j - \vec{r}_i\|^2 + \varepsilon^2)^{3/2}}. \quad (1)$$

Integración temporal (Euler explícito):

$$\vec{v}_i \leftarrow \vec{v}_i + \vec{a}_i \Delta t, \quad (2)$$

$$\vec{r}_i \leftarrow \vec{r}_i + \vec{v}_i \Delta t. \quad (3)$$

4.1. Convención de kernels CUDA

- **Variante básica (obligatoria):** un hilo CUDA por cuerpo i ; cada hilo escribe solo $\vec{a}_i$ y recorre en serial todo $j \neq i$. Mapeo 1D: `i = blockIdx.x * blockDim.x + threadIdx.x`.
- **Variante con memoria compartida (obligatoria):** tiles de posiciones (y masas si aplica) cargados en `__shared__`; `__syncthreads()` entre carga y uso; mismo resultado físico que la variante básica dentro de la tolerancia acordada.
- Orden fijo del paso temporal en host: (1) lanzar kernel de aceleraciones; (2) `cudaDeviceSynchronize()`; (3) actualizar todas las velocidades y posiciones en host con Euler explícito; (4) copiar a device solo si el siguiente paso lo requiere.
- Preferir **SoA** (structure of arrays) en memoria global del device para favorecer coalescing.
- La referencia CPU serial del Lab 1 es la fuente de verdad para regresión.
- Tolerancia de comparación CPU vs. GPU: documentar en README valores `rtol` y `atol` (punto de partida sugerido: `rtol = 1e-4`, `atol = 1e-8` para aceleraciones; ajustar y justificar si usan otro criterio).

Ejemplo numérico. Dos masas en el eje x : m_1 en $x = 0$, m_2 en $x = d > 0$. Sobre la partícula 1, $a_{1,x} = Gm_2 d / (d^2 + \varepsilon^2)^{3/2}$. Con $G = m_2 = d = 1$, $\varepsilon = 0,1$: $a_{1,x} \approx 0,971$. Debe coincidir entre CPU y GPU.

4.2. Fundamentos teóricos breves

Energía cinética $K = \frac{1}{2} \sum_i m_i \|\vec{v}_i\|^2$; energía potencial con el mismo suavizado: $U = -G \sum_{i < j} m_i m_j / \sqrt{\|\vec{r}_j - \vec{r}_i\|}$. Con Euler explícito, $E = K + U$ no se conserva exactamente; muestren la deriva en el reporte.

En GPU, el speedup real depende del cómputo y de las transferencias host-device. Definan en el reporte una fracción serial f que incluya, como mínimo, copias y trabajo en host no paralelizable en el paso medido.

5. Requisitos técnicos CUDA

5.1. Conceptos obligatorios

La implementación debe cubrir, al menos una vez de forma correcta y comprobable:

1. *Modelo de ejecución:* `__global__`, `__device__`, `__host__`; lanzamiento «<grid, block>».
2. *Memoria:* `cudaMalloc`, `cudaMemcpy` (H2D, D2H; D2D opcional), `cudaFree`; punteros device separados de host.
3. *Índices:* cálculo de índice global 1D; grilla con división techo $\lceil N/\text{blockDim.x} \rceil$.
4. *Memoria compartida:* `__shared__`, `__syncthreads()` en la variante optimizada.
5. *Reducción:* kernel o rutina que calcule K , U o una suma global con patrón de reducción en shared memory (puede ser en dos pasadas).
6. *Atómicas:* `atomicAdd` (u otra atómica justificada) en al menos una métrica global.
7. *Errores:* macro `CUDA_CHECK` (o equivalente) tras APIs CUDA y `cudaGetLastError` tras kernels.

8. *Temporización (sin eventos CUDA)*: `std::chrono` en host alrededor de `cudaDeviceSynchronize()` — **no** se exige `cudaEvent_t` en este laboratorio.

5.2. Diseño esperado de los kernels

El enunciado no entrega código base de kernel: cada grupo debe construirlo, justificarlo y probarlo. En el reporte incluyan el mapeo de datos a hilos, el cálculo de `grid/block`, el tratamiento del borde $i \geq N$, el layout de memoria elegido y la razón por la que la variante con `__shared__` es equivalente a la variante básica.

5.3. Métodos obligatorios y sobrecarga

Usen sobrecarga de funciones (como en el Lab 1) para variantes CUDA sin duplicar lógica física:

1. `NBodySystem` (o `NBodySystemGPU`):

- `computeAccelerationsGpu()`
- `computeAccelerationsGpu(int variant)` con 0=básico, 1=shared memory
- `computeAccelerationsGpu(int variant, int block_size)`

2. `NBodySimulator`:

- `stepEulerGpu()`
- `calculateEnergyGpu()`
- `calculateEnergyGpu(int method)` con 0=reducción, 1=`atomicAdd`

3. `Benchmark`:

- `benchmarkKernelOnly()` — tiempo del kernel (sync incluida en la medición host)
- `benchmarkEndToEnd()` — incluye transferencias del paso
- `compareCpuGpu(int n_bodies)`

6. Flujo Git, releases e issues

A partir de este laboratorio el repositorio debe operar como un proyecto de software real.

6.1. Ramas y protección

- La rama `main` (o `master`) debe estar **protegida**: sin push directo; merge solo vía merge request (GitLab) o pull request (GitHub).
- Trabajo en ramas `feature/<nombre>` o `fix/<nombre>`; eliminar ramas fusionadas tras el merge.
- Cada MR/PR debe referenciar al menos un issue (Closes #N, Refs #N o equivalente en GitLab).
- Al menos una revisión humana del equipo antes de fusionar a `main` (además de CI verde).

6.2. Changelog y releases

- Mantener `CHANGELOG.md` siguiendo el formato *Keep a Changelog* (<https://keepachangelog.com/>).
- Al entregar, etiquetar release `v2.0.0-lab2` en `main` con notas que resuman cambios CUDA, CI y agentes.
- Entradas mínimas: `Added`, `Changed`, `Fixed` para la versión 2.0.0.

6.3. Issues

- Los issues los crea el equipo en GitHub/GitLab (no solo el docente y el ayudante). El rol de **Git, releases y agentes** coordina el tablero; el de **Calidad, CI y visualización** revisa que estén bien formulados.
- Mínimo durante el laboratorio: **5 issues** creados por el equipo (al menos 1 por rol distinto), **3 MR/PR** fusionados a `main` y cada MR/PR vinculado a un issue.
- Cada issue debe tener: título claro, descripción, etiquetas (`bug`, `documentation`, `agent`, `cuda`, etc.) y asignado.
- En evaluación se revisarán issues y MR/PR con relación directa a esos issues.

6.4. Ejemplo de flujo

```
1 git checkout main && git pull
2 git checkout -b feature/shared-memory-kernel
3 # ... commits convencionales: feat(cuda): add shared-memory tile kernel
4 git push -u origin feature/shared-memory-kernel
5 # Abrir MR "feat(cuda): shared-memory accelerations (Closes #12)"
6 # CI pasa -> agente MR comenta -> revisión humana -> merge
```

7. Agentes de IA en el repositorio

Inspiración de práctica industrial: agentes que automatizan trabajo **mecánico y repetitivo**, y escalan a humanos lo que requiere juicio (véase el caso de revisión asistida por agentes en proyectos de código abierto¹).

Configuren **tres agentes** usando scripts versionados en el repositorio y un disparador en CI (GitHub Actions, GitLab CI o equivalente). Los prompts y configuraciones deben vivir en `scripts/agents/` o `.github/workflows/` (o ruta equivalente documentada en el README).

7.1. Agente documentador

- **Frecuencia:** al menos una vez por semana o al fusionar cambios en `main`.
- **Acción:** revisar README, comentarios de clase y `CHANGELOG.md`; abrir issues de documentación faltante o desactualizada.

¹<https://simonwillison.net/2026/Jul/5/sqlite-utils-fable/>

- **Si el arreglo es mecánico** (typo, sección faltante con plantilla obvia, enlace roto): el agente puede auto-asignarse el issue, abrir MR con el fix y etiquetar `agent:auto-fix`.
- **Si requiere juicio técnico** (explicar un kernel, decisiones de diseño): comentar en el issue `Requiere intervención humana: <motivo>` sin fusionar solo.

7.2. Agente revisor de bugs

- **Frecuencia:** diaria (cron en CI) sobre `main` o la rama de integración acordada.
- **Acción:** revisar código y tests; buscar regresiones, manejo de errores CUDA ausente, desincronización host/device, tests rotos.
- **Si el fix es mecánico** (falta `CUDA_CHECK`, test que falla por tolerancia mal copiada): abrir MR o issue con parche sugerido.
- **Si afecta física, API pública o lógica de kernels:** abrir issue o comentario `Requiere intervención humana` — no modificar `main` directamente.

7.3. Agente revisor de merge requests

- **Disparador:** al abrir o actualizar un MR/PR, **después** de que el pipeline de CI termine.
- **Acción:** leer el diff y los resultados de CI; clasificar el MR como `mecánico` y `mergeable` o `requiere revisión humana`.
- **Criterios para mecánico:** solo documentación, formato, tests que pasan, sin cambio de semántica física ni de firma pública sin issue asociado.
- **Salida:** comentario en el MR (nunca merge automático a `main`). Si CI falla, el comentario debe indicarlo explícitamente.

7.4. Entregables de agentes

En el reporte y README incluyan:

- Tabla: agente, herramienta, frecuencia, criterio mecánico vs. humano.
- Al menos **dos ejemplos reales** por agente antes de la entrega (issue creado, MR comentado o caso marcado como “requiere intervención humana”).
- Máximo 5 issues automáticos abiertos por agente por semana sin revisión humana.

8. Análisis de rendimiento

8.1. Mediciones (sin `cudaEvent`)

1. Usar `std::chrono::steady_clock` en host; llamar `cudaDeviceSynchronize()` antes y después del tramo medido cuando incluya kernels.
2. Repetir cada experimento al menos diez veces; reportar $\bar{T} \pm \sigma_T$.
3. Comparar: CPU serial (Lab 1), kernel GPU solo, paso end-to-end con transferencias.

4. Speedup $S = T_{\text{CPU}}/T_{\text{GPU}}$ (mismo N , mismos parámetros físicos, mismo binario CUDA cuando corresponda).
5. Ley de Amdahl con fracción serial f que incluya, como mínimo, transferencias host–device y trabajo en host del paso medido.
6. Estudio de `blockDim.x`: para cada variante de kernel, probar 64, 128, 256, 512 y 1024 (omitir solo tamaños que la GPU rechace); reportar tabla y gráfico tiempo vs. `blockDim.x`.

8.2. Matriz de benchmarks obligatoria (clúster DIINF)

Las corridas finales del reporte deben cubrir, como mínimo, la siguiente matriz en el **nodo GPU del clúster DIINF**:

Parámetro	Valores obligatorios
N (cuerpos)	256, 512, 1024, 2000
Variante de kernel	básica (0) y shared memory (1)
<code>blockDim.x</code>	64, 128, 256, 512, 1024 (los válidos en la GPU)
Pasos temporales por corrida	≥ 100 (solo para benchmark de paso; tests de regresión pueden usar menos)
Repeticiones por punto	≥ 10
Mediciones	kernel-only y end-to-end (con transferencias del paso)

Total mínimo: 40 combinaciones por tipo de medición (kernel-only y end-to-end), salvo `blockDim.x` inválidos documentados en el README. Pueden agregar más puntos, pero no omitir los obligatorios.

8.3. Pruebas finales en clúster DIINF

Las mediciones finales de performance deben ejecutarse en el **clúster del DIINF, nodo GPU**. Las pruebas locales sirven para desarrollo y depuración, pero las tablas y gráficos finales del reporte deben salir del nodo GPU del clúster.

En el README documenten, para cada corrida final:

- Nombre del nodo GPU, modelo de GPU, driver, versión de CUDA y flags de compilación (`nvcc --version, nvidia-smi`).
- Comando exacto para compilar y ejecutar benchmarks en el clúster (incluido módulo o script de entorno si aplica).
- Parámetros: N , pasos, `blockDim.x`, variante de kernel, número de repeticiones, tipo de medición (kernel-only o end-to-end).
- Archivos generados: `benchmark_results.dat`, `blockdim_study.dat` y log de ejecución (`cluster_run.log` o equivalente).

Si el clúster no está disponible tras al menos dos intentos documentados (job fallido, ticket o log con fecha), pueden usar una GPU NVIDIA alternativa; deben declarar la desviación en el README y en el reporte, y la nota de benchmarks puede verse afectada.

8.4. Propagación de errores

Misma fórmula que en el Lab 1. Para $S_p = T_1/T_p$,

$$\sigma_{S_p} = S_p \sqrt{\left(\frac{\sigma_{T_1}}{T_1}\right)^2 + \left(\frac{\sigma_{T_p}}{T_p}\right)^2}. \quad (4)$$

9. Estructura del proyecto

```
nbody_2d_cuda/  
|-- main.cpp  
|-- Particle.h/.cpp  
|-- NBodySystem.h/.cpp  
|-- NBodySimulator.h/.cpp  
|-- Integrator.h/.cpp  
|-- MetricsCalculator.h/.cpp  
|-- Benchmark.h/.cpp  
|-- Visualizer.h/.cpp  
|-- CudaBuffer.h          # RAII cudaMalloc/cudaFree  
|-- kernels/  
|   |-- accelerations.cu/.cuh  
|   |-- integration.cu/.cuh  # opcional  
|   `-- metrics.cu/.cuh  
|-- tests/  
|-- scripts/agents/         # o .github/workflows/  
|-- Dockerfile  
|-- CHANGELOG.md  
|-- Makefile  
`-- README.md
```

10. Parámetros de simulación

Para **tests y regresión**: $N \leq 32$, pocos pasos, semilla fija documentada.

Para **benchmarks finales en clúster**: usar la matriz de la sección anterior ($N \in \{256, 512, 1024, 2000\}$). Parámetros físicos coherentes con el Lab 1: Δt pequeño y estable; $G = 1$ en unidades adimensionales declaradas en README; $\varepsilon \in [10^{-2}, 10^{-1}]$.

11. Análisis y visualización

11.1. Gráficos requeridos

1. Speedup GPU vs. CPU frente a N .
2. Tiempo kernel-only vs. end-to-end (impacto de transferencias).
3. Tiempo frente a `blockDim.x` para ambas variantes de kernel (básica y shared memory), medido en el nodo GPU del clúster DIINF.
4. Curva de Amdahl: predicción vs. medición.
5. Trayectorias o $E(t)$ (como en Lab 1).
6. Comparación básica vs. shared memory (mismo N , mismos parámetros).

11.2. Archivos de salida

- `benchmark_results.dat`, `scaling_analysis.dat`, `blockdim_study.dat`
- `cluster_run.log` (o equivalente con nodo, fecha y comandos)

- `performance_plots.png`
- `trajectories.dat` o `snapshots.dat`
- `energy_timeseries.dat`

12. Pruebas automáticas

- Unitarias: aceleración entre 2–3 cuerpos CPU vs. analítico; GPU vs. CPU con tolerancia.
- Integración: pocos pasos, semilla fija; regresión GPU.
- CI: `make test` obligatorio en cada MR/PR; el pipeline debe incluir al menos compilación y tests CPU. Un job con GPU en CI es opcional; las mediciones finales de performance no se aceptan desde CI, solo desde el clúster DIINF.
- Performance final: scripts reproducibles para ejecutar la matriz de N , variante de kernel y `blockDim.x` en el nodo GPU del clúster DIINF.
- Los agentes no sustituyen la suite de tests: deben ejecutarse antes de cualquier comentario de agente en MR.

13. Instrucciones de compilación

13.1. Makefile (referencia)

```

1 NVCC = nvcc
2 CXX  = g++
3 NVCCFLAGS = -O3 -std=c++17 -Xcompiler -Wall,-Wextra
4 CXXFLAGS  = -Wall -Wextra -O3 -std=c++17
5 LDFlags   = -lcudart
6
7 TARGET = nbody_2d_cuda
8 CPP_SOURCES = main.cpp Particle.cpp NBodySystem.cpp ...
9 CU_SOURCES  = kernels/accelerations.cu kernels/metrics.cu
10
11 $(TARGET): $(CPP_SOURCES) $(CU_SOURCES)
12     $(NVCC) $(NVCCFLAGS) -o $(TARGET) $(CPP_SOURCES) $(CU_SOURCES) $(LDFlags)
13
14 test:
15     ./run_tests
16
17 .PHONY: clean benchmark test

```

13.2. Docker

Imagen base sugerida: `nvidia/cuda:12.x-devel-ubuntu22.04`. Documentar requisito de GPU NVIDIA y driver. El contenedor debe compilar y ejecutar `make test` (tests CPU al menos).

14. Criterios de evaluación

14.1. Implementación CUDA (30 %)

- Kernels básico y shared memory correctos y equivalentes dentro de tolerancia
- Manejo de memoria device y errores CUDA
- Reducción y `atomicAdd` donde corresponda
- SoA y convenciones documentadas

14.2. Git, issues y releases (15 %)

- `main` protegida; mínimo 5 issues y 3 MR/PR fusionados, cada MR vinculado a un issue
- `CHANGELOG.md` y tag `v2.0.0-lab2`
- Historial de commits y MR revisable

14.3. Agentes de IA (10 %)

- Tres agentes configurados y documentados
- Evidencia de uso útil (issues/MR/comentarios)
- Criterio claro mecánico vs. intervención humana

14.4. Pruebas y CI (10 %)

- `make test`; CI en MR; Docker CUDA

14.5. Benchmarks y análisis (20 %)

- CPU vs. GPU; kernel vs. end-to-end; estudio de `blockDim.x`
- Mediciones finales ejecutadas en el nodo GPU del clúster DIINF, con comandos y ambiente reproducibles
- Amdahl y propagación de errores

14.6. Visualización y reporte (15 %)

- Gráficos requeridos; interpretación física y de rendimiento
- Reporte hasta 10 páginas (diseño CUDA, Git, agentes, resultados)

15. Entregables

1. Repositorio Git con historial, `main` protegida, issues y MR/PR del período del lab.
2. Código C++/CUDA, `tests/`, `CHANGELOG.md`, tag `v2.0.0-lab2`.
3. Configuración de los tres agentes en el repo.
4. `Makefile`, `Dockerfile`, CI funcionando.
5. `README.md` actualizado (GPU, Git, agentes, roles).
6. Tablas `.dat`, logs o scripts de ejecución en el clúster DIINF y figuras PNG.
7. Reporte técnico PDF (máx. 10 páginas) con: diseño CUDA y mapeo a hilos; flujo Git y agentes; matriz de benchmarks en clúster; resultados (speedup, `blockDim.x`, Amdahl); conclusiones y limitaciones.

16. Calendario y fecha de entrega

Publicación del enunciado: lunes 6 de julio de 2026.

Entrega: viernes 24 de julio de 2026, 23:59.

Formato: igual que en el Laboratorio 1 (Google Classroom; código comprimido + PDF del reporte).

16.1. Calendario sugerido

Ventana oficial: del **lunes 6 de julio** al **viernes 24 de julio de 2026** (tres semanas). Fechas orientativas por hito:

1. **6–8 jul:** issue de planificación, proteger `main`, `CHANGELOG.md`, layout SoA y buffers CUDA; baseline CPU serial verificado.
2. **9–11 jul:** kernel básico de aceleraciones; tests con $N = 2$, $N = 3$ y semilla pequeña; primer MR fusionado.
3. **12–14 jul:** kernel con `__shared__`; equivalencia CPU/GPU dentro de tolerancia; MR vinculado a issue.
4. **15–17 jul:** `make test`, Docker, CI y los tres agentes con al menos una ejecución registrada cada uno.
5. **18–20 jul:** scripts de benchmark; primeras corridas en clúster DIINF; ajuste de `blockDim.x`.
6. **21–22 jul:** matriz completa de benchmarks en nodo GPU del clúster; tablas `.dat` y `cluster_run.log`.
7. **23 jul:** gráficos, análisis Amdahl, revisión cruzada por roles, release candidate.
8. **24 jul (23:59):** tag `v2.0.0-lab2`, reporte PDF y entrega en Classroom.

17. Recursos adicionales

- CUDA Toolkit: <https://developer.nvidia.com/cuda-toolkit>
- CUDA C++ Programming Guide: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- Material del curso: clase GPGPU ([parallelism/classes/new/gpgpu/](https://github.com/parallelism-classes/new/gpgpu/))
- Nsight Compute (opcional): <https://developer.nvidia.com/nsight-compute>
- Keep a Changelog: <https://keepachangelog.com/>
- Ejemplo de revisión con agentes: <https://simonwillison.net/2026/Jul/5/sqlite-utils-fable/>

18. Consejos de implementación

- Verifiquen $N = 2$ y $N = 3$ en CPU antes de confiar en GPU.
- Minimicen `cudaMemcpy` por paso: mantengan estado en device entre pasos del integrador.
- Empiecen con kernel básico correcto; después agreguen shared memory y comparen.
- No optimicen con Nsight hasta que los tests CPU vs. GPU pasen.
- Los agentes deben ser acotados: mejor pocos hallazgos útiles que muchos issues duplicados.
- Documenten en README la GPU y driver usados en benchmarks.

19. Recordatorios

- Compilar sin advertencias (`-Wall -Wextra`).
- No usar `cudaEvent_t` para las mediciones obligatorias; usar `std::chrono + cudaDeviceSynchronize()`.
- Conservar baseline CPU serial del Lab 1.
- Cada concepto CUDA obligatorio debe tener ruta verificable en test o benchmark.
- Los MR sin issue asociado pueden penalizarse en la rúbrica de Git.
- Los agentes nunca fusionan a `main` sin humano.
- En el test de salida pueden pedirles que expliquen un issue abierto por un agente y un kernel con `__shared__`.